

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное образовательное учреждение высшего образования
САНКТ – ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

Старший преподаватель

должность, уч. степень, звание

подпись, дата

С.Ю. Гуков

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №4

Стек с поддержкой максимума

по курсу: Алгоритмы и структуры данных

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков

инициалы, фамилия

Санкт-Петербург 2024

СОДЕРЖАНИЕ

1 Цель работы	3
2 Задание	4
3 Краткое описание хода работы.....	6
4 Исходный код	7
5 Результат работы с примерами	11
6 Выводы	12

1 Цель работы

Цель данной лабораторной работы заключается в реализации стека с поддержкой операций `push`, `pop` и `max`, а также дополнительных операций, таких как `min`, `avg` и других, с обеспечением константного времени выполнения всех операций. Это позволяет изучить и понять основные принципы работы с абстрактными структурами данных и эффективностью различных операций.

2 Задание

Второй пункт лабораторной работы, описывающий задание, имеет целью реализацию стека с поддержкой операций `push`, `pop` и `max`. Стек, как абстрактная структура данных, позволяет организовать хранение данных таким образом, чтобы последние добавленные элементы были доступны первыми. Это свойство называется принципом LIFO (Last In, First Out).

Для выполнения задания необходимо реализовать функциональность, которая обеспечит добавление элементов в стек с помощью операции `push`. Эта операция должна добавлять элемент на верхушку стека, при этом важно учитывать ограничение на максимальный размер стека. Если стек уже заполнен, операция `push` должна возвращать ошибку или игнорировать запрос.

Кроме того, необходимо реализовать операцию `pop`, которая удаляет верхний элемент стека. Эта операция также должна возвращать значение удалённого элемента. Если стек пуст, необходимо предусмотреть обработку такой ситуации, чтобы избежать ошибок при попытке удалить элемент из пустого стека.

Ключевой задачей является реализация операции `max`, которая возвращает максимальное значение, находящееся на стеке. Эта операция должна работать за константное время, что потребует использования вспомогательной структуры данных. Для этого целесообразно использовать дополнительный стек, который будет хранить текущие максимальные значения на каждом этапе добавления элементов.

Дополнительно к основным операциям, требуется реализовать две из предложенных дополнительных операций. Одной из них может стать `min`, которая будет возвращать минимальное значение на стеке. Вторая дополнительная операция может быть `avg`, которая будет вычислять среднее значение всех элементов в стеке. Все операции должны оставаться эффективными и выполняться за константное время.

Задание ставит перед собой задачу не только реализации базового функционала стека, но и разработки более сложных операций, что потребует

от разработчика понимания работы со структурами данных и их взаимодействия. Такой подход позволит углубить знание об абстрактных структурах данных и научиться оптимизировать алгоритмы для обеспечения высокой производительности.

Варианты дополнительных задач.

- Реализовать min, avg за константное время,
- Реализовать ограничение размера стека,

3 Краткое описание хода работы

В третьем пункте лабораторной работы описан ход разработки стека. Первоначально был выбран язык программирования JavaScript для реализации структуры данных.

Разработка началась с создания класса Stack с основными методами: push, pop и max. Операция push включает проверку на максимальный размер стека, чтобы избежать переполнения. Метод pop удаляет верхний элемент и проверяет, не является ли стек пустым.

Для реализации операции max был использован вспомогательный стек, который хранит текущие максимальные значения, что позволяет выполнять эту операцию за константное время. В дальнейшем были добавлены операции min и avg, которые отслеживают минимальные значения и вычисляют среднее значение элементов в стеке.

Тестирование проводилось для проверки корректности работы всех функций. Все операции были реализованы в соответствии с заданием и успешно протестированы.

4 Исходный код

Ниже предоставлен исходный код моей программы с комментариями основных блоков:

```
class Stack {
  constructor(maxSize) {
    this.stack = [];
    this.maxStack = [];
    this.minStack = [];
    this.sum = 0;
    this.count = 0;
    this.maxSize = maxSize;
  }

  push(value) {
    if (this.count >= this.maxSize) {
      console.error('Стек переполнен. Невозможно добавить элемент.');
```

return;

```
    }
    this.stack.push(value);
    this.sum += value;
    this.count++;

    if (this.maxStack.length === 0 || value >= this.maxStack[this.maxStack.length
- 1]) {
      this.maxStack.push(value);
    }

    if (this.minStack.length === 0 || value <= this.minStack[this.minStack.length
- 1]) {
      this.minStack.push(value);
```

```

    }
}

pop() {
    if (this.count === 0) {
        console.error('Стек пуст. Невозможно удалить элемент.');
```

return;

```
    }
    const poppedValue = this.stack.pop();
    this.sum -= poppedValue;
    this.count--;

    if (poppedValue === this.maxStack[this.maxStack.length - 1]) {
        this.maxStack.pop();
    }

    if (poppedValue === this.minStack[this.minStack.length - 1]) {
        this.minStack.pop();
    }

    return poppedValue;
}

max() {
    return this.maxStack[this.maxStack.length - 1];
}

min() {
    return this.minStack[this.minStack.length - 1];
}

```



```

    avg() {
      if (this.stack.length === 0) return 0;
      return this.sum / this.count;
    }
  }
}

```

// Ввод данных с клавиатуры

```

const readline = require('readline').createInterface({
  input: process.stdin,
  output: process.stdout
});

```

```

readline.question('Введите максимальный размер стека: ', (maxSize) => {
  const myStack = new Stack(parseInt(maxSize));

```

```

  readline.question('Введите количество запросов: ', (q) => {
    let queries = 0;

```

```

    const processQuery = () => {
      if (queries < q) {
        readline.question('Введите запрос (push v, pop, max, min, avg): ',
(input) => {
          const parts = input.split(' ');
          const command = parts[0];

```

```

          switch (command) {
            case 'push':
              const value = parseInt(parts[1]);
              myStack.push(value);

```

```

        break;
    case 'pop':
        myStack.pop();
        break;
    case 'max':
        console.log('Максимум:', myStack.max());
        break;
    case 'min':
        console.log('Минимум:', myStack.min());
        break;
    case 'avg':
        console.log('Среднее значение:', myStack.avg());
        break;
    default:
        console.log('Неверная команда');
    }
    queries++;
    processQuery(); // Запрос следующей команды
});
} else {
    readline.close();
}
};

processQuery(); // Запускаем обработку запросов
});
});

```

5 Результат работы с примерами

Тестирование функций stack, q, maxStack, push, avg.

```
Введите максимальный размер стека: 4
Введите количество запросов: 3
Введите запрос (push v, pop, max, min, avg): push 3
Введите запрос (push v, pop, max, min, avg): push 4
Введите запрос (push v, pop, max, min, avg): avg
Среднее значение: 3.5
```

Рисунок 1 – Результат первого тестирования

Тестирование остальных функций.

```
Введите максимальный размер стека: 7
Введите количество запросов: 7
Введите запрос (push v, pop, max, min, avg): push 3
Введите запрос (push v, pop, max, min, avg): push 4
Введите запрос (push v, pop, max, min, avg): pop 4
Введите запрос (push v, pop, max, min, avg): max
Максимум: 3
Введите запрос (push v, pop, max, min, avg): push 1
Введите запрос (push v, pop, max, min, avg): min
Минимум: 1
Введите запрос (push v, pop, max, min, avg): avg
Среднее значение: 2
```

Рисунок 2 – result.txt

6 Выводы

В процессе выполнения лабораторной работы была достигнута основная цель — реализация стека с поддержкой операций `push`, `pop` и `max`, а также дополнительных функций, таких как `min` и `avg`. Каждая из этих операций была спроектирована так, чтобы обеспечивать максимальную эффективность и производительность, что позволило успешно справиться с требованиями задания.

Проведенное тестирование подтвердило, что все операции работают корректно и эффективно. Использование вспомогательных структур для хранения максимального и минимального значений, а также аккумулярование суммы элементов в стеке обеспечили выполнение операций за константное время. Это продемонстрировало высокую производительность алгоритма и его возможность масштабирования для работы с большими объемами данных.

Кроме того, лабораторная работа позволила глубже понять принципы работы со структурами данных и научиться оптимизировать алгоритмы для достижения высокой производительности. Это знание будет полезным при разработке более сложных приложений и систем, требующих эффективного управления данными.

В итоге, полученные результаты и опыт, приобретенный в ходе работы, открывают новые горизонты для дальнейшего изучения и применения стековых структур в различных задачах программирования. Работа с такими структурами является важной частью подготовки к разработке более сложных алгоритмов и систем, что делает её неотъемлемой частью учебного процесса.